

Juan Andrés Mejía Ortiz

Federico Bohorquez

Informe desafio 1

Para este desafío, nosotros hicimos un pre informe en el que plasmamos lo que pensábamos hacer, en este dijimos como pensábamos hacer las fichas, los movimientos y el tablero, pero no dijimos mucho del código porque no estábamos seguros de cómo hacerlo. Los módulos los terminamos planteando cuando ya empezamos, terminamos haciendo 3 si no contamos los headers. Tampoco se especificó que el líder del repositorio en github iba a ser Juan Andrés. Todo esto se nos olvidó decirlo en el preinforme, pero si lo hicimos en nuestro código y lo estamos poniendo en este informe final. Tenemos ganas de hablar de cómo nos fue en el proceso del desafío y de cómo resolvimos los problemas cuando se nos presentaron, que no fueron pocos.

Al inicio todo iba bien, empezamos por el tablero.h y .cpp, esto lo hicimos con lo que dijimos en el preinforme, las dimensiones y como pensábamos hacer la identificación de las fichas. Este desafío lo separamos para poder hacerlo más eficientemente, pero solo al inicio. Aparte de la identificación de fichas y de la dimensión del tablero, también se hizo la función de liberación de memoria de este mismo al inicio, luego se hizo el “guardado” de fichas, que simplemente es representar la ficha luego de que se identificó su posición en el tablero.

También se planteó las funciones que podía hacer el jugador, más específicamente las de agregar y eliminar filas/columnas. Todo eso fue fluyendo bastante bien, el qt no nos decía que habían problemas que tuviéramos que resolver y estuvimos muy pendientes de problemas de fugas de memoria o algún otro, pero hubo un momento en el que si nos salieron “falsos positivos” de errores de memoria, pues el qt hace una especie de prevención por si no se toman las precauciones suficientes con el código, para que no se desborde la memoria o se salga de la estática, estos errores eran del analizador estático de Clang, esto lo hace si nota que pueden haber desbordamientos, como ya habíamos dicho. Pero luego de investigar lo suficiente nos dimos cuenta de que esto no era un problema real, pues nuestro acceso de memoria estaba bien estructurado y los desplazamientos eran correctos, haciendo que las advertencias no fueran importantes, por lo tanto pudimos continuar sin problema..

Para la siguiente parte del desafío, se empezó a realizar el juego.h y el .cpp, en este sí se encontraron ciertos problemas (porque en medio de la realización si habían ciertas funciones que no cumplían los parámetros o simplemente habían pequeños errores en

estas que había que corregir). Aparte de esto, si había un error en el `tablero.cpp`, se movía directamente hacia este nuevo archivo, pues como ya sabemos, el código no es inteligente y hace lo que uno le pida, así esté mal en su procedimiento, por lo que tuvimos que cambiarlas ligeramente (y todo esto lo dijimos en el github, para aclarar los cambios que fuimos haciendo con las funciones y los archivos). Continuando con el tema de qué hicimos en el `juego.cpp`, este fue más que todo el movimiento de las fichas en el tablero (las cascadas) y las jugadas que podía realizar el jugador aparte de la redimensión del tablero. También hicimos las funciones de mostrar el tablero en binario, con las fichas y con la numeración. Este lo hicimos en formato de matriz, para que el jugador hiciera las jugadas de manera precisa.

Al momento de hacer las pruebas de integración dentro del main sí se encontraron bastantes problemas, pues no funcionaba correctamente alguna función y hacía un efecto mariposa, dañando el resto del funcionamiento del juego o simplemente no hacía lo que se esperaba, pues al tener errores cuando guardamos los bits de la ficha, este hacía que en el módulo del juego no se hicieran correctamente las cosas. También podía pasar con la parte del tablero, que si en este había problemas con las filas o columnas, el módulo del juego no iba a servir correctamente, pues estos están directamente enlazados. Pero al final lo pudimos resolver, ya entre los dos, para poder tener el desafío listo en el tiempo esperado.

Nos dimos cuenta en medio de la ejecución del juego que las fichas que propusimos en el preinforme no eran las adecuadas, porque realmente era complicado diferenciarlas, en especial porque nuestro tablero está diseñado con símbolos de + y -, y en nuestras fichas teníamos el mismo símbolo, por lo que nos tocó cambiarlos y terminamos asignándoles unos que pensamos que se ven bien. Intentamos también poner varios símbolos como por ejemplo el °, pero este no está dentro del sistema ASCII, por lo que no lo toma como un carácter y hace que se salga del valor de memoria de 1 solo bit, resultando en un error.

Para terminar, vamos a explicar nuestros algoritmos y tareas que implementamos en el código.

Primero, empezamos por la representación de los datos en bits. Acá definimos las fichas, la codificación en bits de estas y los estados que se nos dijo en el desafío. También se hicieron los algoritmos para la búsqueda y empaquetado de las fichas.

Segundo, hicimos la estructura del tablero, haciendo la creación dinámica y la liberación de este, luego hicimos las funciones de redimensión del tablero, incluyendo la regla de la memoria al 65% y asegurándonos de que los bits que teníamos no se destruyeran o movieran de una manera inadecuada.

Tercero, hicimos la lógica del juego y los movimientos de las fichas dentro del tablero. Se hicieron las funciones de detección de combinaciones, eliminación de ficha (que es una de las jugadas del usuario) y las cascadas, que son las que hacen que caigan las fichas, por lo tanto, rellenando los huecos.

Por último hicimos la integración de todo lo anterior, dándole una estructura bonita visualmente e implementando todas las funciones ya creadas para poder probar el programa, y cuando había errores, se solucionaban. También se hicieron los formatos de visualización del tablero, tanto de forma binaria, como con las fichas.

Ahora, para los algoritmos (no vamos a decir todos, pero sí los más importantes):

Empezamos con el `tablero.cpp`, en el cuál tenemos lo más importante del desafío, que es guardar las fichas.

`guardarFicha` es el nombre de esta función y en esta teníamos que calcular el bit inicial (que el nombre en el código es `bit_inicial`), luego determinar el byte buscado (`byte_buscado`) y el desplazamiento del bit en este (`desplazamiento_bit`), luego creamos una máscara (`bitmask`) & para poder limpiar solo los 3 bits en el byte, mientras que se conservan los otros 5 bits, luego se combinaba con el operador OR = (`|=`) el valor de nuestra ficha desplazada.

Ahora con `eliminarColumna`. En esta calculamos el tamaño físico necesario para poder redimensionar, luego creamos un arreglo dinámico temporal para poder ir respaldando nuestras fichas (esto lo hicimos porque antes de aplicar esto se nos estaban perdiendo, creando muchos problemas de memoria). Luego se recorre la matriz para posición que no sea de la columna a eliminar, para poder usar `obtenerFicha` (que es otra función dentro del código) y guardarla en este arreglo dinámico temporal, después actualizamos el tamaño físico del tablero (que es cuando la memoria es $<65\%$). Después de hacer esto, se recorre el arreglo dinámico temporal y se usa `guardar ficha` para escribir la matriz nuevamente en el tablero nuevo con la columna agregada, y por último liberamos el arreglo temporal.

Este último algoritmo es también muy importante para el juego y es el de `ejecutarCascadas`. En este se inicializa un arreglo dinámico de booleanos para poder identificar las casillas a eliminar, luego se inicia el bucle del `while` para ejecutar la función de `detectarYEliminar`. Si en esta se detectan combinaciones, se marcan como vacío en el tablero, luego se aplica la función de `aplicarGravedad` y se ejecuta la de `rellenarVacios`. Esto se repite siempre que el `detectarYEliminar` sea verdadero, pues pueden seguir habiendo combinaciones, por lo tanto, cascadas.